Agent Used: University secure Copilot
Purpose: Brainstormed and decide on an Idea

Prompt [Let brain storm ideas fro this project a) Minimum... | Try in Copilot Chat]

Let brainstorm ideas for this project. a) Minimum Technical Requirements To achieve a Pass (i.e. 40+), your API must at a minimum: • implement at least one data model involving all four elements of CRUD (Create, Read, Update and Delete) functionality linked to a database.2 You can base your design on a RESTful (Representational State Transfer) API. However, you can also use a GraphQL-based API if you think this is more suitable for the problem, but you must explain the rationale for using GraphQL. • includes at least four API endpoints accessible via HTTP requests • handle user inputs and return appropriate JSON responses. You are permitted to use other formats, provided you justify these in your supporting report document. • use correct status/error codes, according to industry convention • be demonstrable via local execution, web hosting (e.g. Pytho- nAnywhere), or a Model Context Protocol (MCP) server. You can base your implementation on Django, but you can also use other suitable stacks (e.g. Python FastAPI, Node.js Express, Go Fiber). You must justify your technical choices and motivations. (b) Example Project Ideas • A book metadata and recommendation API integrating a public dataset (e.g. Goodreads or Google Books) and providing analytic endpoints such as genre trends, rating distributions, and person- alised suggestions. • An environmental and urban climate statistics API that stores his- torical weather, air quality, and temperature data, enabling users to query aggregated metrics, anomalies, and city-level trends. • A productivity and habit analytics API that tracks tasks, habits, time logs, and completion patterns, offering endpoints for streak calculations, productivity heatmaps, and behavioural summaries. 2You are permitted to use any type of SQL database, but you should not use NoSQL databases except with clear reasoning in your supporting report document. 7 • A public transport reliability and delay analytics API using open UK transport datasets, providing CRUD for user-reported inci- dents and endpoints for route-level reliability scores, delay patterns, and temporal performance summaries. • A nutrition and recipe analytics API backed by open food datasets (e.g. USDA or UK FSA), supporting ingredient/recipe CRUD and computing calories, macronutrients, allergen warnings, and cost or difficulty estimates. • A housing market and rental insights API integrating ONS or Land Registry data, offering CRUD for listings and locations, plus analytics such as median rent, affordability indices, or regional price trends. • A cultural events and tourism insights API using public datasets on museums, festivals, and heritage sites, enabling CRUD for events and providing popularity metrics, seasonal trends, and location-based recommendations. • A sports performance and match statistics API backed by public sports datasets (e.g. football, cricket, or F1), supporting CRUD for teams, players, and matches, alongside analytical endpoints for leaderboards, performance summaries, and win probabilities

Response

Absolutely — here are strong, well-scoped project ideas that meet all the minimum requirements and give you plenty of room for CRUD, analytics, and justification of technical choices. I'll give you:

1. 5 polished project concepts
2. What CRUD model(s) they could use
3. Suggested endpoints
4. Potential analytics features
5. Why they make sense for Django / FastAPI / Express

🔥 Top Brainstormed Ideas for Your API Project

---

1. Smart Fitness & Workout Analytics API

A system for tracking gym workouts, exercises, muscle groups, and progress trends.

CRUD Model

- Workout (Create logs, update them, delete, view)
- Exercise (Create new exercises, update details, etc.)

Endpoints

- POST /workouts/ – create workout
- GET /workouts/{id} – read workout
- PUT /workouts/{id} – update workout
- DELETE /workouts/{id} – delete workout
- GET /analytics/progress?user=X – show strength/volume progression
- GET /analytics/heatmap?user=X – training frequency heatmap

Analytics Ideas

- Volume trends (sets × reps × weight)
- Exercise frequency
- Estimated 1RM calculations
- Training load spikes (injury risk indicator)

Why this project is great

Easy to model relationally, and analytics gives you an opportunity to write meaningful logic.

---

2. Sustainable Shopping & Carbon Footprint API

Users track items they buy; the API calculates carbon impact using open datasets.

CRUD Model

- Product
- Purchase
- Brand

Endpoints

- POST /products/ – create product
- GET /products/ – list
- POST /purchases/ – user logs purchase
- GET /analytics/carbon?user=X – total CO_2 impact
- GET /analytics/categories – category-level breakdown

Analytics Ideas

- CO_2 usage over time

- **High-impact vs low-impact purchases**
- **Category-level comparisons (food, clothes, electronics)**

Why this project works

Perfect for REST; clean relationships; real-world data available.

---

3. Student Revision & Learning Insights API

Tracks revision sessions, topics, and generates learning analytics.

CRUD Model

- **Topic**
- **RevisionSession**

Endpoints

- **POST /session/ – log revision**
- **GET /session/{id}**
- **GET /analytics/streaks – revision streak tracking**
- **GET /analytics/productivity – hour-of-day productivity**

Analytics Ideas

- **Revision streaks**
- **Productivity heatmap**
- **Weak vs strong topics (based on time spent)**

Tech justification

FastAPI or Django fit perfectly. Could even justify using GraphQL due to nested queries (topic → sessions).

---

4. Cinema & Streaming Recommendation API

Users CRUD movies and reviews; system computes personalised suggestions.

CRUD Model

- **Movie**
- **Review**
- **User watchlist**

Endpoints

- **POST /movie/**
- **POST /review/**
- **GET /analytics/ratings – rating distributions**
- **GET /recommendations?user=X – simple recommendation engine**

Analytics Ideas

- **Genre distribution**
- **Highest-rated in last X days**
- **Collaborative filtering (simple version)**

Why this is strong

You can justify REST or GraphQL, and the dataset availability makes it easy.

---

5. Inventory & Price Optimisation API (Retail Analytics)

Tracks store inventory and uses simple analytics to identify trends.

CRUD Model

- **Item**
- **PriceHistory**

Endpoints

- POST /item/
- PUT /item/{id}
- GET /analytics/price-trend/{item} – shows price movement
- GET /analytics/low-stock – items running out

Analytics

- Low-stock detection
- Price trend graphs
- Average monthly price

Good for marks

This is extremely easy to implement fully, meets all requirements, and looks professional.

⭐ Simple But High-Scoring Project Idea (Recommended)

Public Transport Delay API (Mini Version of Your Example)

Instead of integrating huge datasets, make it lightweight:

CRUD Model

- Incident (User-reported delays)
- Route
- TransportService

Endpoints

- /incidents/ – CRUD
- /analytics/reliability?route=X
- /analytics/hourly-delay-pattern

Analytics

- Average delays per route
- Peak hours
- Most unreliable services

Very easy to justify Django or FastAPI, and data modelling is well-defined.

🎯 Want me to help you pick one and write the whole project plan?

I can also generate:

✔ full API architecture

✔ data models (SQL schema)

✔ endpoint designs

✔ example JSON

✔ justification for framework choice (FastAPI/Django/Express)

✔ explanation suitable for your report

Just tell me which idea you're leaning toward — or I can shortlist my top 3 for marks.

Prompt more brain storming lets step away from the... | Try in Copilot Chat
more brain storming lets step away from the exmaples im intresteed in transport, housing and sports

Response
Absolutely — let's go deeper and brainstorm new, original API project concepts based on your three interest areas: transport, housing, and sports.
None of these appear to require web search, so I won't search unless you ask.
I'll give you creative, unique, API-friendly, CRUD-friendly ideas with potential models, endpoints, and analytics hooks.

🚇 Transport – Fresh API Concepts
1. Traffic Flow & Congestion Insight API
Track road segments, user-reported congestion, journey times, and generate analytics.
Possible Models
- **RoadSegment**
- **JourneyLog**
- **CongestionReport**

Analytics Ideas
- **Predictive congestion (based on time of day)**
- **"Best time to travel" estimator**
- **Road reliability score**
- **Congestion heatmap (hourly/daily)**

---

2. Micro-Mobility Usage API (E-scooter / Bike Hire Simulator)
Model micro-mobility usage and trends—like Lime/Bird-style analytics but simplified.
Possible Models
- **Vehicle (bike/scooter)**
- **Trip**
- **Zone (parking areas / geofenced zones)**

Analytics
- **Zone popularity**
- **Average trip distance**
- **Vehicles needing maintenance (based on mileage)**
- **Carbon savings compared to car**

---

3. Taxi Fare & Route Optimisation API
Simulate journeys, fares, driver ratings, demand hotspots.
Possible Models
- **Driver**
- **TaxiRide**
- **FareEstimate**
- **PickupPoint**

Analytics
- **Surge pricing predictor**

- Most profitable zones
- Driver efficiency score
- Ride distance distribution

---

4. EV Charging Station Availability API
Track charging stations, user-reported faults, and usage patterns.
Possible Models
- ChargingStation
- ChargingSession
- FaultReport

Analytics
- Peak charging times
- Slow vs rapid charger performance
- Fault frequency ranking
- "Charging desert" detection (underserved areas)

🏠 **Housing – New & Interesting API Concepts**
5. Home Energy Efficiency & Cost Insights API
Focus on EPC-like energy ratings and cost analytics.
Models
- Property
- EnergyAssessment
- ImprovementRecommendation

Analytics
- Estimated energy cost savings
- Carbon footprint per property
- Heat-loss estimation models
- Seasonal heating costs

---

6. Tenant–Landlord Issue Tracking & Resolution API
A "housing issues" case management system with analytics.
Models
- Property
- IssueReport (plumbing, damp, heating)
- RepairJob

Analytics
- Average issue resolution time
- Most common housing faults
- Property "health score"
- Landlord responsiveness metrics

---

7. Housing Accessibility & Mobility Audit API
Great niche idea — for evaluating accessibility features.
Models
- Property
- AccessibilityFeature (steps, ramps, door widths)
- AuditReport

Analytics

- **Accessibility score per property**
- **Improvements required**
- **Compliance with guidelines (basic scoring model)**
- **City-level accessibility maps (conceptual)**

---

8. House Move Planner & Inventory API
Tracks inventory, packing progress, moving tasks, and cost estimates.
Models

- **InventoryItem**
- **Room**
- **MovingTask**
- **SupplierService (mover, cleaning, packing)**

Analytics

- **Packing progress timelines**
- **Cost breakdowns**
- **Weight/volume estimates for van size**
- **Task priority suggestions**

⚽ **Sports – Fresh Concepts**
9. Athlete Performance & Fatigue Load API
Track training sessions + fatigue scoring.
Models

- **Athlete**
- **TrainingSession**
- **FatigueScore**

Analytics

- **Training load calculation (e.g., TRIMP-like scoring)**
- **Fatigue vs performance curves**
- **Overtraining risk alerts**
- **Performance trend graphs**

---

10. Amateur League Management API
Allow team managers to CRUD teams, fixtures, results, and leaderboards.
Models

- **Team**
- **Player**
- **Match**
- **Result**

Analytics

- **Live league table**
- **Form guide (last 5 matches)**
- **Player performance index**
- **Win probability modelling (simple Elo)**

---

11. Running Route Performance & Weather Insight API
Link run logs with weather conditions.

Models

- **Run**
- **Route**
- **WeatherSnapshot (localised per run)**

Analytics

- **Pace vs temperature correlation**
- **Best-performing routes**
- **Weather impact score**
- **Improvement rates over time**

---

12. Fantasy Sports Draft & Stats API

Track players, drafts, points, and match performance.

Models

- **Player**
- **DraftTeam**
- **MatchStat**
- **GameWeek**

Analytics

- **Points predictors**
- **"Best value players"**
- **Injury risk score**
- **Player momentum index**

🧠 **Want even more niche ideas?**

Here are some really unique API concepts combining your interests:

🚇 + ⚽ **Transport to Sports Venue Travel Times API**

- **Predict travel time to stadiums on match days**
- **Crowd-level congestion scoring**
- **Best travel route recommendations**

🏠 + ⚽ **Housing + Sports: "Home Fitness Layout Optimiser" API**

- **CRUD room dimensions + equipment**
- **Suggest optimal layouts for workouts**
- **Time-of-day heatmap for home noise levels (light simulation)**

🏠 + 🚇 **Housing + Transport: Commute-Score API**

- **"Commute friendliness score" for any property**
- **Nearest transport links**
- **Delay histories for local stations**

👋 **Want me to help you pick the strongest one?**

I can help you choose by comparing:

✅ **complexity**

✅ **originality**

✅ **how easy it is to implement CRUD**

✅ **how strong it looks for academic marking**

Just tell me what excites you most: transport, housing, sports, or a hybrid idea.

Prompt [for sports look inot f1 conscepts and transport... | Try in Copilot Chat]
for sports look inot f1 conscepts and transport lookingto flight and trains

Response
Perfect — let's deep-dive into brand-new API concepts specifically for:

- 🏎️ F1 (sports)
- ✈️ Flights (transport)
- 🚆 Trains (transport)

All ideas will be API-friendly, CRUD-ready, and rich with analytics endpoints so your project meets the requirements and scores well.

🏎️ F1-Focused API Concepts (original + technically rich)
1. F1 Race Telemetry & Performance Insight API
A simplified version of F1 telemetry systems (like AWS F1 Insights), but academic-friendly.
CRUD Models

- Driver
- CarSetup (aero, tyres, wings)
- LapTime
- RaceSession

Analytics Endpoints

- GET /analytics/optimal-lap?driver=X
  → Computes ideal lap combining fastest sectors.
- GET /analytics/tyre-degradation?driver=X
  → Estimates tyre performance drop per lap.
- GET /analytics/pace-trend?session=qualifying
  → Pace evolution over a session.
- GET /analytics/battle-simulator?driver1=A&driver2=B
  → Compare pace deltas to predict overtakes.

Why it's strong

- Feels like a real F1 engineering tool.
- CRUD is natural (drivers, setups, laps).
- Analytics = big marks.

---

2. F1 Constructor Development Tracker
Simulates mid-season development upgrades.
CRUD Models

- Team
- UpgradePackage
- PerformanceMetric (aero efficiency, engine power)
- RaceResult

Analytics

- Upgrade impact forecasting
- "Development race" leaderboard
- Budget-cap efficiency ratios
- Predicted finishing positions

---

3. F1 Strategy & Pitstop Planner API
Focus on tyre compounds, pit windows, and race simulation.
CRUD Models

- **Strategy**
- **Stint**
- **TyreCompound**
- **PitStopEvent**

Analytics

- **Optimal pit stop strategy generator**
- **Undercut/overcut effectiveness scoring**
- **Race time estimator**
- **Tyre wear curves**

---

4. F1 Fan Engagement & "Race Moments" API
Focus on fan engagement rather than engineering.
CRUD Models

- **RaceMoment (overtake, crash, penalty, fastest lap)**
- **FanRating (crowd upvotes)**
- **Driver**

Analytics

- **Most exciting race moments**
- **Driver excitement index**
- **Moment heatmaps (lap vs intensity)**

✈️ Flight-Related API Concepts (transport)
5. Flight Delay Prediction & Monitoring API
A mini version of a real airport operations tool.
CRUD Models

- **Flight**
- **Airport**
- **DelayReport**
- **WeatherSnapshot**

Analytics

- **Delay probability estimator**
- **Route reliability score**
- **"Worst airports for delays"**
- **Weather–delay correlation**

---

6. Airline Carbon & Efficiency API
Focus on sustainability modelling.
CRUD Models

- **AircraftType**
- **FlightRecord**
- **FuelBurnEstimate**

Analytics

- **CO_2 estimation per flight**
- **Efficiency comparison by aircraft**

- **Fuel burn vs distance curves**
- **Greener alternatives suggestions**

---

7. Passenger Baggage Flow & Lost-Bag Simulation API
A quirky but *very unique* idea.
CRUD Models
- **Bag**
- **BaggageTransferEvent**
- **Flight**
- **AirportZone**

Analytics
- **Lost-bag risk estimator**
- **Transfer efficiency**
- **Airport "baggage reliability" score**

🚆 **Train-Related API Concepts (transport)**
8. Real-Time Train Punctuality & Delay Pattern API
A powerful analytics-based project.
CRUD Models
- **TrainService**
- **Station**
- **DelayReport**
- **JourneyLog**

Analytics
- **Average route punctuality**
- **"Most delayed stations"**
- **Time-of-day reliability curves**
- **Seasonal delay patterns**

---

9. Train Crowd Prediction & Capacity API
Simulates crowding levels on trains.
CRUD Models
- **TrainCarriage**
- **PassengerCount**
- **Station**
- **JourneySegment**

Analytics
- **Crowding prediction for future times**
- **Carriage utilisation heatmap**
- **Busiest stations**
- **Passenger flow modelling**

---

10. Rail Maintenance & Fault Monitoring API
Focus on equipment and infrastructure.
CRUD Models
- **TrackSection**
- **MaintenanceReport**

- FaultType
- InspectionRecord

Analytics

- Failure risk score
- Maintenance priority ranking
- Fault frequency trend
- Predictive maintenance model

🔥 Hybrid Ideas (F1 + Flight + Train Themes)

11. "Speed & Efficiency Comparison API"

Compare modes of transport (F1 car vs plane vs train) in three dimensions:

- Energy usage
- Emissions
- Time efficiency

Amazing for a sustainability-focused twist.

---

12. "Race Travel Logistics Planner"

Track logistics for moving F1 equipment (planes, trucks, trains) between race venues.

Models:

- Shipment, Vehicle, Route, CustomsDelay

Includes analytics for travel time, delays, and cost.

⭐ Want help narrowing down the best one?

Tell me which direction you prefer:

- Engineering-heavy (telemetry, strategy)
- Analytics-heavy (delays, performance modelling)
- Sustainability-focused
- Fan/community-driven
- Hybrid creative concepts

I can then produce:

✔ Final project concept
✔ Database models
✔ Full endpoint list
✔ Example JSON
✔ Justification for tech stack (Django/FastAPI)
✔ Research-ready explanation for your report

Which one should we develop into a full spec?

Prompt [f1 number 1 is good idea lets exand this inot a... | Try in Copilot Chat]
f1 number 1 is good iead lets exand this inot a paln

Response
Awesome — let's turn **F1 Project Idea #1 (Telemetry & Performance Insight API)** into a **full, well-structured project plan** suitable for implementation *and* for writing your academic report.
This plan will include:
✔ Project overview
✔ Technical justification (framework, database, architecture)
✔ Data model design (with tables + fields)
✔ API endpoint specification
✔ Analytics logic (explained clearly)
✔ Example JSON
✔ Error-handling + status codes
✔ Stretch goals (optional extras for higher marks)

🏎️ F1 Telemetry & Performance Insights API — Full Project Plan

---

1. Project Overview
The goal is to build an **F1 telemetry and race analytics API** that allows users to:
- Create and store data about drivers, setups, laps, and sessions
- View lap telemetry and performance metrics
- Run analytics such as:
  - Optimal lap calculation
  - Tyre degradation modelling
  - Pace trend analysis
  - Driver comparison ("who is faster and where?")

This mimics real F1 engineering tools but simplified for an academic API project.

---

2. Technical Stack Justification
Framework: FastAPI (recommended)
Why?
- Extremely fast to build REST endpoints
- Automatic OpenAPI docs (/docs)
- Async support for performance
- Cleaner than Django for API-first projects
- Great for JSON-based responses
- Easy validation using Pydantic

(You can justify Django REST Framework if preferred — I can write that too.)
Database: SQLite or PostgreSQL
- SQLite is sufficient for local execution and academic marking
- SQL DB required by assignment spec, fits relational F1 structure

Architecture
- RESTful API
- CRUD for all models

- Analytics endpoints as pure functions reading DB data
- Clear separation:
  - /drivers/
  - /laps/
  - /sessions/
  - /analytics/...

---

3. Data Model Design
Here's a clean relational schema.

3.1 Driver

Field	Type	Notes
id	int (PK)	
name	string	
team	string	e.g. Ferrari
country	string	optional

3.2 RaceSession

Field	Type	Notes
id	int	PK
name	string	FP1 / FP2 / Quali / Race
track	string	e.g. Silverstone
date	datetime	

3.3 CarSetup
(One setup per driver per session)

Field	Type	Notes
id	int	PK
driver_id	FK → Driver	
session_id	FK → RaceSession	
wings	int	e.g. 7
tyre_compound	string	Soft/Medium/Hard
fuel_load	float	kg
ride_height	float	mm

3.4 LapTime

Field	Type	Notes
id	int	PK
driver_id	FK → Driver	
session_id	FK → Session	

Field	Type	Notes
lap_number	int	
sector1	float	seconds
sector2	float	
sector3	float	
tyre_life	int	laps on tyre at that lap

---

4. API Endpoint Specification
4.1 CRUD Endpoints
Drivers

- GET /drivers/
- POST /drivers/
- GET /drivers/{id}
- PUT /drivers/{id}
- DELETE /drivers/{id}

Race Sessions

- POST /sessions/
- GET /sessions/{id}
- (and full CRUD)

Car Setups

- POST /setups/
- GET /setups/{id}

Lap Times

- POST /laps/
- GET /laps/?session=X&driver=Y
- PUT /laps/{id}
- DELETE /laps/{id}

---

5. Analytics Endpoints
These give the project depth and marks.

5.1 Optimal Lap Time
Endpoint:
GET /analytics/optimal-lap?driver_id=X&session_id=Y
Logic:
Pick the **best sector 1**, **best sector 2**, **best sector 3** for that driver and sum them.
Example result:

```
{
  "driver_id": 1,
  "optimal_lap": 82.435,
  "best_sectors": {
    "sector1": 23.112,
    "sector2": 29.554,
    "sector3": 29.769
```

```
  }
}
```

5.2 Tyre Degradation Model
Endpoint:
GET /analytics/tyre-degradation?driver_id=X&session_id=Y
Logic:
Lap time plotted vs tyre life → estimate degradation rate via simple linear regression.
Response example:

```
{
  "driver_id": 1,
  "avg_degradation_per_lap": 0.145,
  "predicted_lap_time_on_fresh_tyres": 82.10
}
```

5.3 Pace Trend
Endpoint:
GET /analytics/pace-trend?session_id=Y&driver_id=X
Returns lap-by-lap times with moving averages.

5.4 Driver Comparison
Endpoint:
GET /analytics/compare?driver1=A&driver2=B&session_id=X
Outputs deltas:

```
{
  "driver1": "Leclerc",
  "driver2": "Sainz",
  "average_delta": -0.152,
  "fastest_laps": {
    "driver1": 81.554,
    "driver2": 81.706
  }
}
```

---

6. Status Codes & Error Handling
Examples:

- 201 CREATED → when a new driver or lap is created
- 400 BAD REQUEST → invalid sector values
- 404 NOT FOUND → driver or session missing
- 409 CONFLICT → setup already exists for that session
- 500 INTERNAL SERVER ERROR → analytics error

---

7. Example JSON for Creating a Lap

```
{
  "driver_id": 3,
  "session_id": 10,
  "lap_number": 14,
```

"sector1": 23.551,
"sector2": 29.773,
"sector3": 29.410,
"tyre_life": 6
}

---

8. Stretch Goals (if you want a distinction)

✔ **DRS usage modelling**
✔ **Weather influence**
✔ **Real telemetry channels (speed, throttle, brake)**
✔ **Race strategy simulator**
✔ **GraphQL endpoint justification (optional)**
✔ **JWT authentication system (lightweight)**

🔥 **Next Step Options**

I can now generate **any of these**:

✅ **Full implementation file structure**
✅ **SQL schema ready to run**
✅ **API documentation in Markdown (for your report)**
✅ **UML diagram of data models**
✅ **FastAPI skeleton code**
✅ **Sample dataset (drivers + laps)**

What would you like next?